Syntax Analyzer

Kedar Sedai

***Abstract*— This document describes the syntax analysis phase of the MiniLang compiler, in which a recursive-descent parser processed tokens produced by the lexical analyzer and generates an abstract syntax tree for subsequent semantic processing.**

I. INTRODUCTION

The syntax analyzer (parser) is compiler phase 2. It checks that tokens form valid MiniLang structure and builds an Abstract Syntax Tree(AST).

Source Code (.minilang)
↓
Lexer (Phase 1)
↓
Token Stream
`// kind, lexeme, location, int_value`
↓
Parser (Phase 2)
↓
AST (Program)
`// tree of Expr, Stmt, FunctionalDecl`

The parser does not re-scan characters. It only reads tokens from the lexer via Lexer::next_token().

The minilang_parser tool wires lexer -> parser -> AST dump: parser_main.cpp

```
const std::string source =
    read_file(path);
minilang::Lexer lexer(source, path);
if (lexer.had_error()) {
    return 2;
}
minilang::Parser parser(lexer);
const minilang::Program program =
    parser.parse();
minilang::dump_program(std::cout,
    program);
```

the steps is

- Reads the source file into a string.
- Creates `Lexer` (produces tokens on demand).
- Creates `Parser(lexer)` — reads tokens and builds the AST.
- Calls `parse()` → returns `Program`.
- Prints the tree using `dump_program()`.

II. PARSING STRATEGY: HAND-WRITTEN RECURSIVE DESCENT

MiniLang uses a recursive-descent parser:

Property	Detail
`Type`	Top-down, one grammar rule, one function
`Lookahead`	1 token(current)
`Output`	AST nodes as C++ structs

TABLE I: MiniLang recursive-descent parser:

Each grammar rule has a function e.g. parse_function(), parse_if_statement(), parse_expression(). Function call each other recursively to match nested syntax(blocks inside if, calls inside expression etc)

III. HOW TOKENS ARE CONSUMED

The parser keeps one current token and uses helper methods:

```
Lexer& lexer_;
  Token current_;
  bool had_error_ = false;
  bool panic_mode_ = false;
```

Constructor - prime the parser

```
Parser::Parser(Lexer& lexer) :
   lexer_(lexer) {
   advance();
}
```

On creation, the parser reads the first token so current_ always holds the next token to interpret.

IV. CORE HELPERS

Method	Purpose
`advance()`	current=$lexer.nexttoken(); reportlexerInvalidtokens$
`check(kind)`	is current.$kind === kind?(peek, noconsume)$
`match(kind)`	if kind matches, consume and return true
`consume(kind, msg)`	Require token or } report parse error
`error(...)`	Print error with filename:line:column, set had$_error$
`synchronize()`	Skip tokens until ; or } after a bad declaration

TABLE II: Core Helpers

Streaming model: tokens are pulled one at a time; the full token list is not required in memory

A MiniLang file is a sequence of functions. Each function starts with a type keyword(int, bool, void). Function parsing parse_function():

1) `parse_type()` → return type
2) Expect Identifier → function name
3) `{` → optional `parse_parameters()` →)
4) `parse_block()` → body

Example tokens for int factorial(int n) ...

KW_INT → IDENT(factorial) → LPAREN → KW_INT → IDENT(n) → RPAREN → LBRACE → ...

Statements: parse_statement() Uses first-token dispatch (like a switch or current_.kind):

```
std::unique_ptr<Stmt>
   Parser::parse_statement() {
   if (check(TokenKind::KwInt) ||
      check(TokenKind::KwBool)) {
      return parse_var_decl();
   }

   if (check(TokenKind::KwIf)) {
      return parse_if_statement();
   }

   if (check(TokenKind::KwWhile)) {
      return parse_while_statement();
   }

   if (check(TokenKind::KwReturn)) {
      return parse_return_statement();
   }

   if (check(TokenKind::LBrace)) {
      return std::make_unique<Stmt>(
         std::move(*parse_block())
      );
   }

   // ... expression statement
}
```

First token	Statement type
int, bool	variable declaration
if	if/else
while	while loop
return	Return
{	Nested block
anything else	Expression statement (expr;)

V. Expressions:precedence climbing via recursion

parse_expression() delegates to a chain of functions, lowest precedence first:

```
parse_expression()
   parse_logical_or()      ||
      parse_logical_and()     &&
         parse_equality()    == !=
            parse_comparison()  < <= > >=
               parse_term()       + -
                  parse_factor()    * / %
                     parse_unary()     ! -
                        parse_call()
                           parse_primary()
```

Precedence (low -> high) || -> && -> == != -> < <= > >= -> + - -> * /

Example: a < b && b != 0

```
          &&
         /  \
        <    !=
       / \   / \
      a   b b   0
```

Each while (check(...)) loop builds left-associative binary nodes:

```
std::unique_ptr<Expr>
   Parser::parse_logical_or() {
   std::unique_ptr<Expr> expr =
      parse_logical_and();
   while (check(TokenKind::OrOr)) {
      SourceLocation loc =
         current_.location;
      advance();
      std::unique_ptr<Expr> right =
         parse_logical_and();
      BinaryExpr bin;
      bin.location = loc;
      bin.op = BinaryOp::Or;
      bin.left = std::move(expr);
      bin.right = std::move(right);
      expr =
         std::make_unique<Expr>(std::move(bin));
   }
   return expr;
}
```

Primary expressions and function calls parse_primary() handles atoms:

Token(s)	AST node
IntLiteral	IntLiteralExpr (uses token.int$_value$)
true/false	BoolLiteralExpr
Identifier	VariableExpr, or CallExpr if (follows
(expr)	parenthesized sub-expression

TABLE III: Primary expression and function calls

VI. AST structure

The parser constructs trees defined in include/minilang/ast/ast.hpp

Hierarchy

```
Program
  FunctionDecl[]
       return_type, name
       Param[]
       BlockStmt (body)
            Stmt[]
                 VarDeclStmt
                 IfStmt (condition, then, optional
                 WhileStmt
                 ReturnStmt
```

```
ExprStmt
BlockStmt (nested)
```

VII. Worked example: return n * factorial(n - 1);

A. Tokens

KW_RETURN IDENT(n) STAR IDENT(factorial) LPAREN IDENT(n) MINUS INT(1) RPAREN SEMICOLON

B. Parse steps

```
parse_return_statement() - sees
   KW_RETURN
parse_expression() -> ... ->
   parse_factor()

Left: VariableExpr(n) from IDENT
STAR -> binary mul

Right: parse_unary()
     -> parse_primary()
     -> IDENT factorial + (

finish_call() - argument
   parse_expression()
          -> n - 1 as BinaryExpr

Expect ;
```

C. AST(conceptual)

ReturnStmt
BinaryExpr (*)
VariableExpr(n)
CallExpr(factorial)
BinaryExpr(-)
VariableExpr(n)
IntLiteralExpr(1)

VIII. Error handling

Two error sources: Lexical - invalid token from lexer (Invalid)
Syntax - unexpected token (e.g. missing ; bad structure)

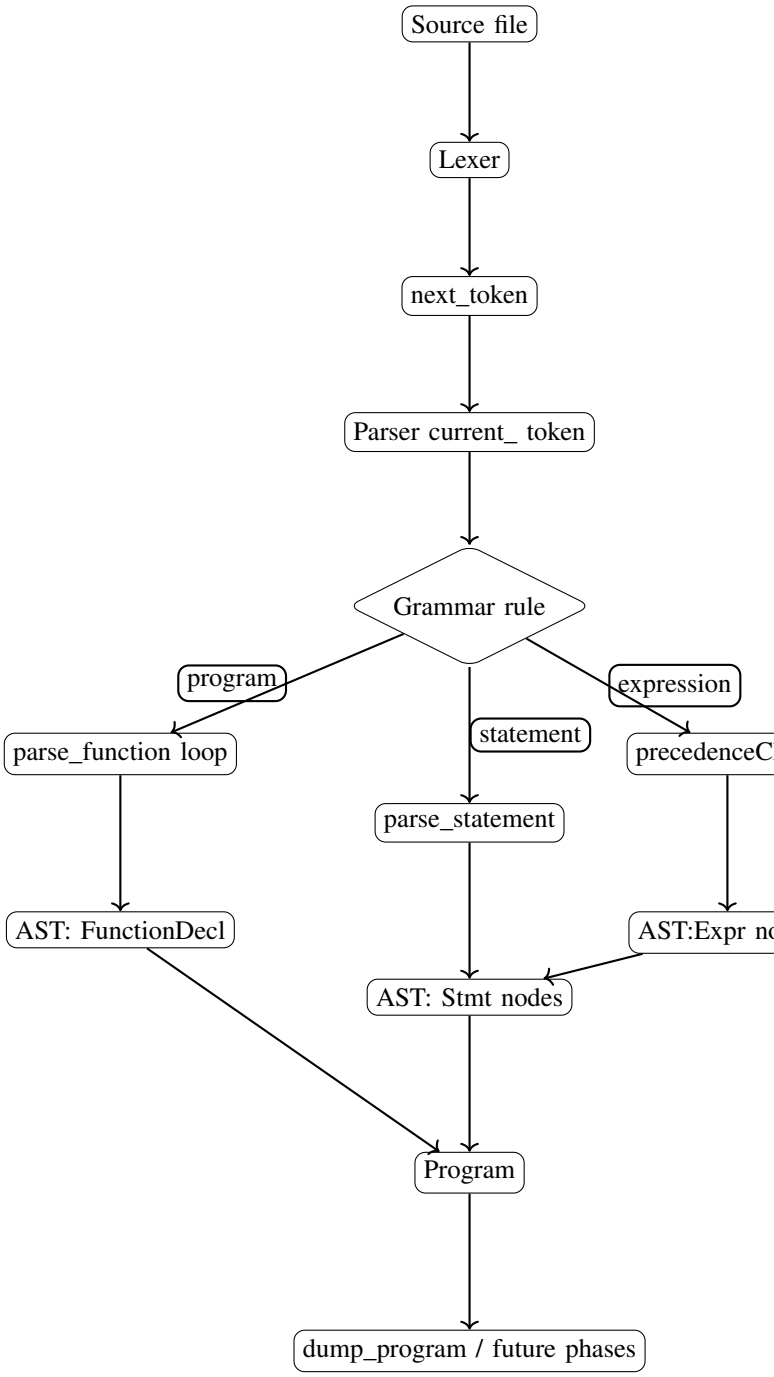
```
void Parser::error(const
   SourceLocation& loc, const
   std::string& message) {
   if (panic_mode_) {
      return;
   }
   panic_mode_ = true;
   had_error_ = true;
   std::cerr << "parse error at " <<
      loc.to_string() << ": " <<
      message << '\n';
}
```

```
panic_mode_ -> only one error message
   per recovery region

synchronize() -> after top-level
   failure,
            skip to next ; or }

parser does not perform type checking
(e.g. bool vs int) - this is in phase 3
```

IX. Flowchart



X. Conclusion

The Syntax Analyzer successfully converts the lexer's token stream into a structured Abstract Syntax Tree for MiniLang. Using a hand-written recursive-descent parser, it recognizes functions, statements, control-flow, and expressions with correct operator precedence, while attaching source locations for error reporting. The AST provides a clear, tree-based representation of the program for the next compiler phase, development can proceed to semantic analysis, where types, scopes and meaning will be validated before intermediate code generation.